

Ex: 1	Private Cloud using OpenStack
Date:	

Aim

To create a private cloud environment by installing and configuring OpenStack (MicroStack) on a single Ubuntu system. The experiment also helps in learning how cloud resources such as virtual machines, networking, and storage are managed in a private cloud.

What is Openstack?

OpenStack is an open-source cloud platform used to build and manage cloud infrastructure. It is mainly used for Infrastructure as a Service (IaaS), where users can create and control virtual machines whenever required. It provides services for computing, networking, and storage in a cloud environment.

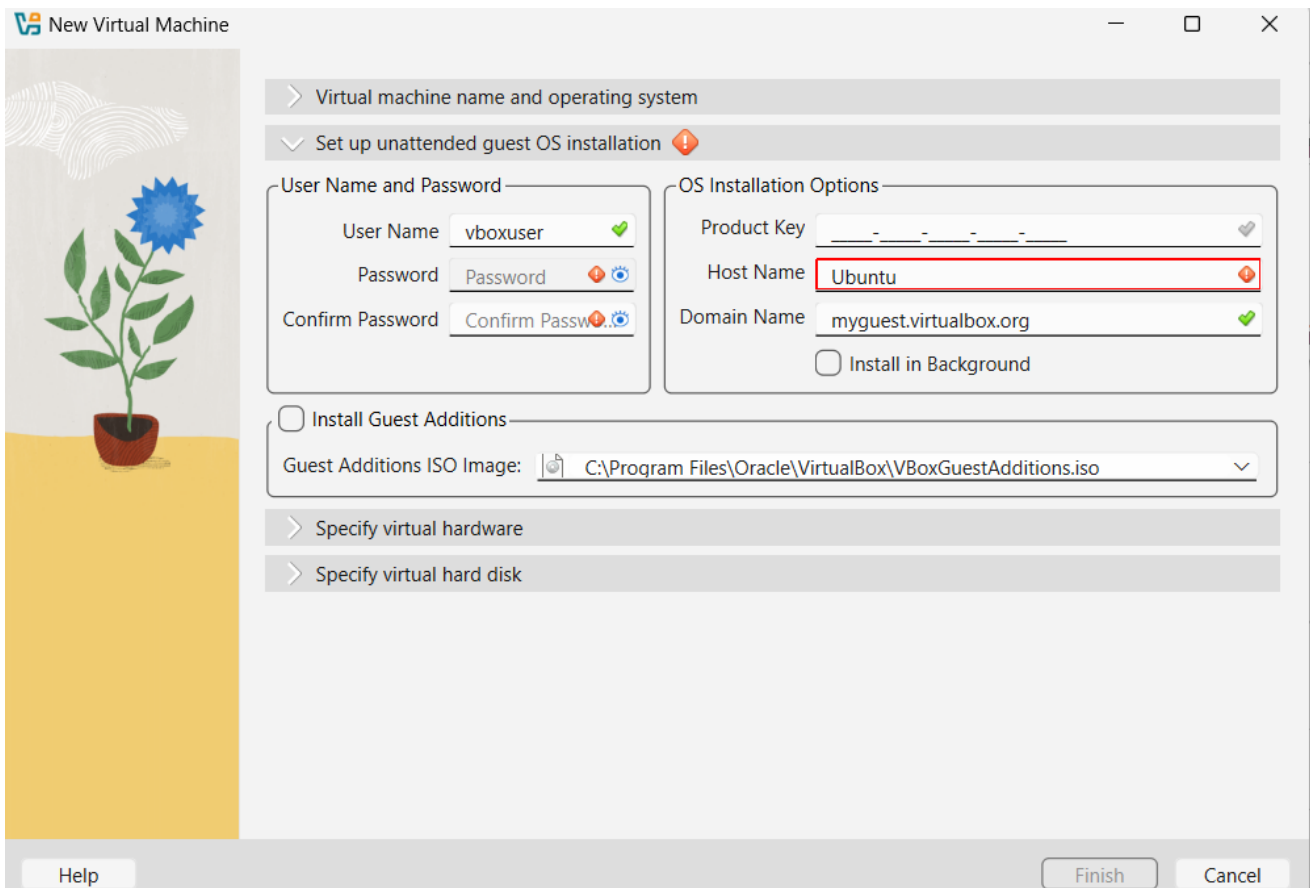
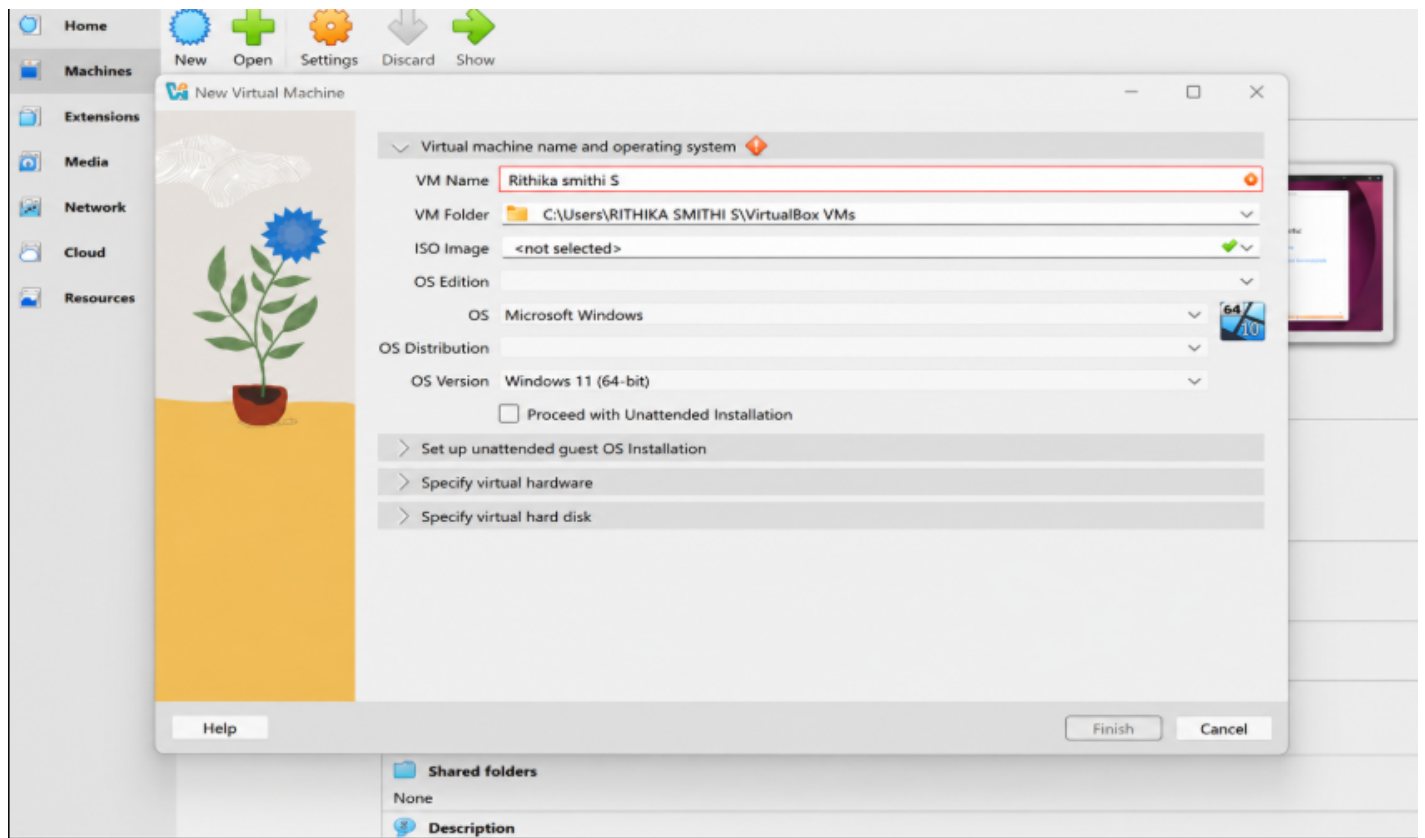
What is Microstack?

MicroStack is a lightweight version of OpenStack that runs on a single computer. It is mainly used for learning, testing, and small-scale development. Since it requires only minimal configuration, it is a good choice for beginners who want to understand how OpenStack works.

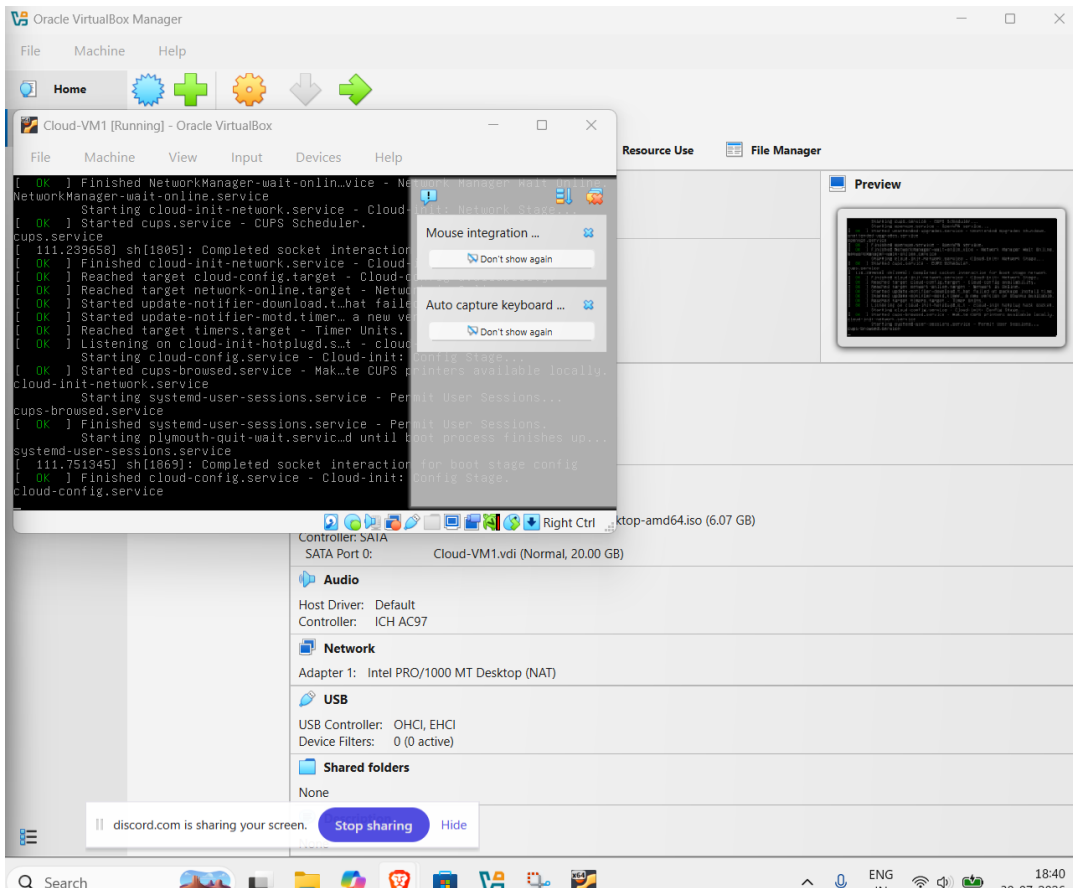
Procedure

Step 1: Create a new Ubuntu virtual machine using Oracle VirtualBox with the following configuration:

- Memory: 8 GB RAM
- Processor: 4 CPU Cores
- Storage: 20 GB Virtual Disk



Step 2: Install the Ubuntu operating system and log in using the username and password created during installation.

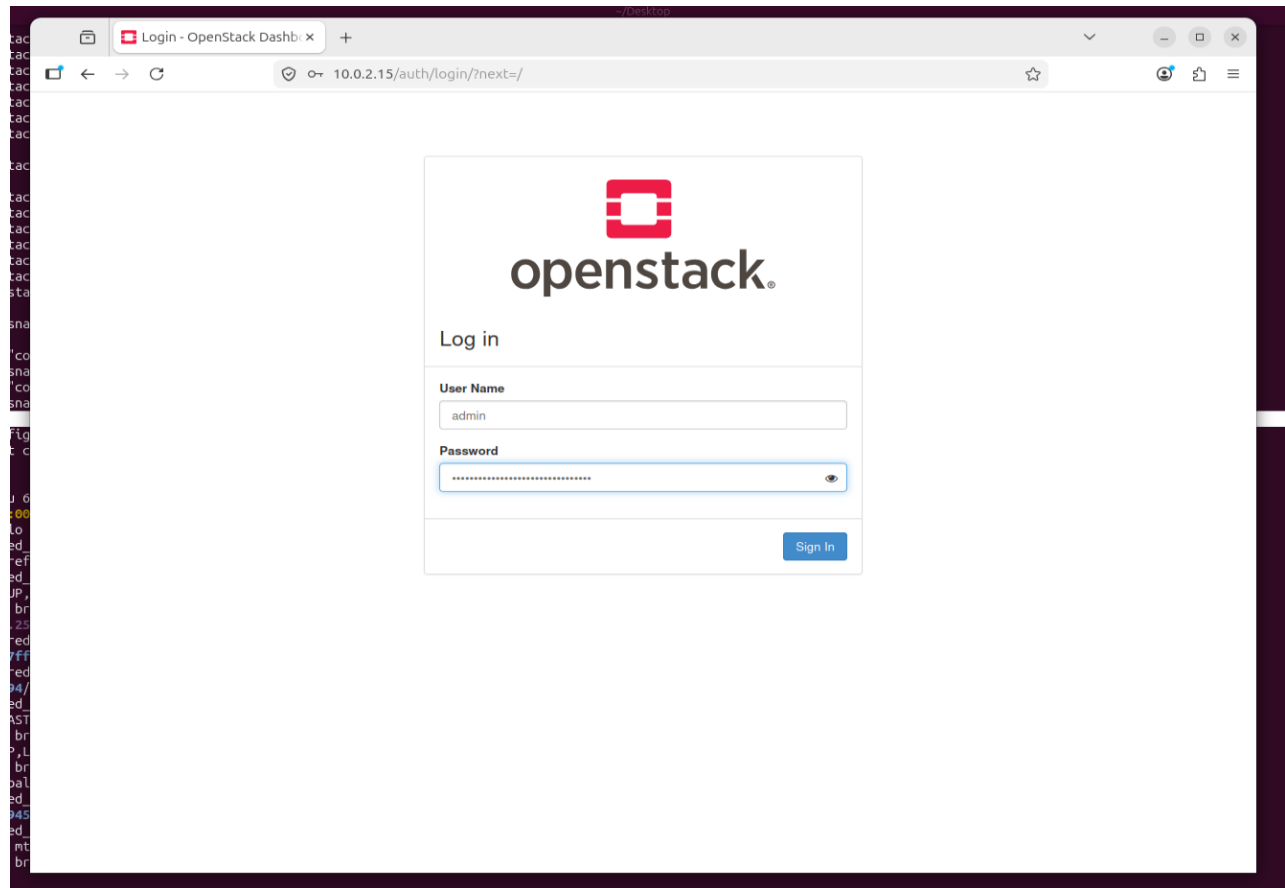


Step 3: Update the Ubuntu packages: `sudo apt update ; sudo apt upgrade`

Step 4: Install MicroStack (Beta channel): `sudo snap install microstack --beta`

Step 5: Check whether MicroStack has been installed successfully: `snap list microstack`

Step 6: Initialize the OpenStack services automatically by executing the following command: `sudo microstack init --auto --control`

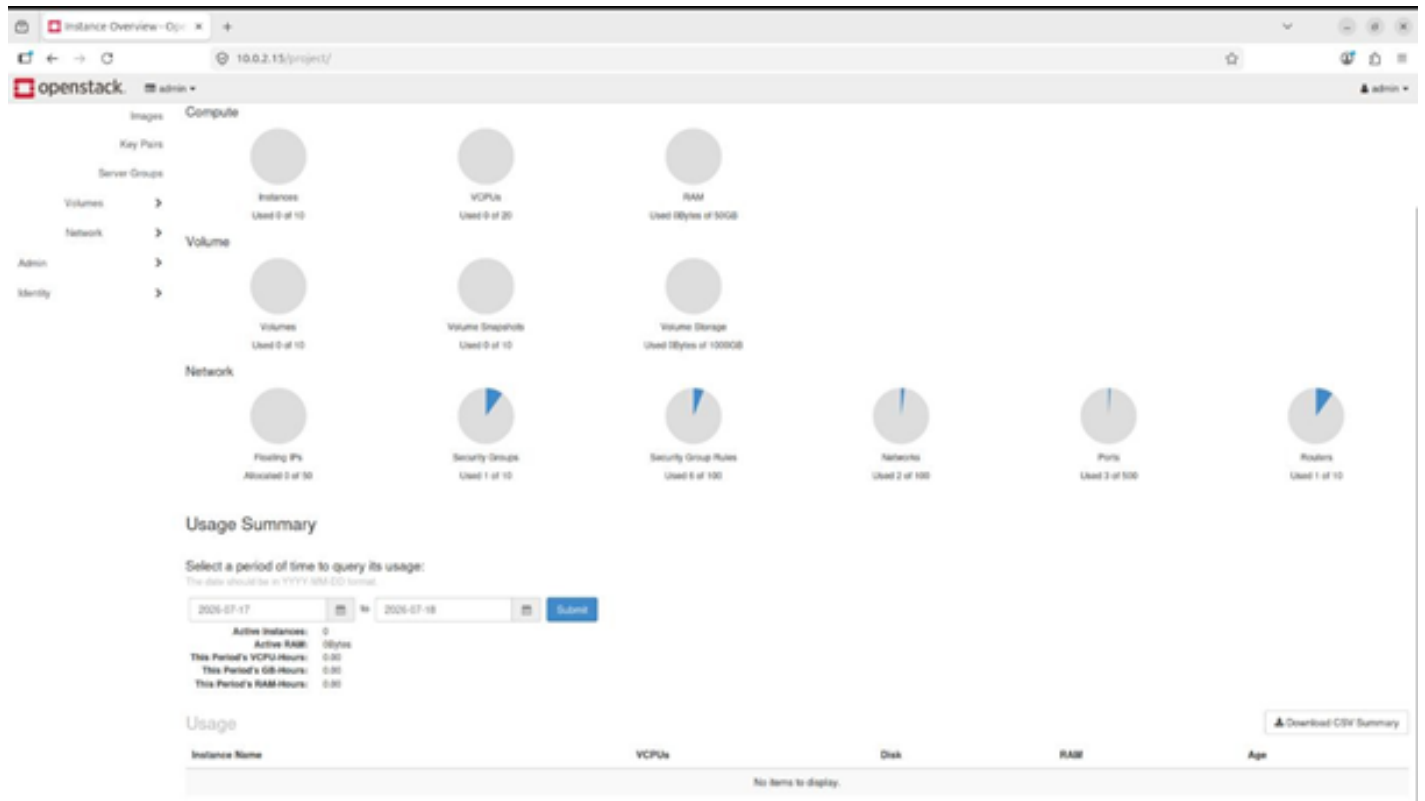


Step 7: Verify the installed OpenStack version: `microstack.openstack --version`

Step 8: Retrieve the Keystone administrator password: `sudo snap get microstack config.credentials.keystone-password`

Step 9: Open the OpenStack Horizon Dashboard in the web browser using the IP address provided after initialization and log in using:

- Username: admin
- Password: (obtained from the previous command)



Conclusion

OpenStack (MicroStack) was successfully installed and configured on Ubuntu, and the Horizon Dashboard was accessed.

Ex: 2

Date:

PaaS in Public Cloud Using Render

Aim

To build a basic Python Flask web application and deploy it on the Render cloud platform. This experiment helps in understanding how a Platform as a Service (PaaS) allows applications to run online without manually setting up or maintaining servers.

What is PaaS?

Platform as a Service (PaaS) is a cloud service that provides everything needed to develop, deploy, and run an application. It takes care of server management, software installation, and deployment, allowing developers to focus only on writing the application.

Why Render?

Render is an easy-to-use cloud hosting platform that supports web application deployment directly from GitHub. It automatically builds and hosts the application, making deployment simple even for beginners.

Procedure

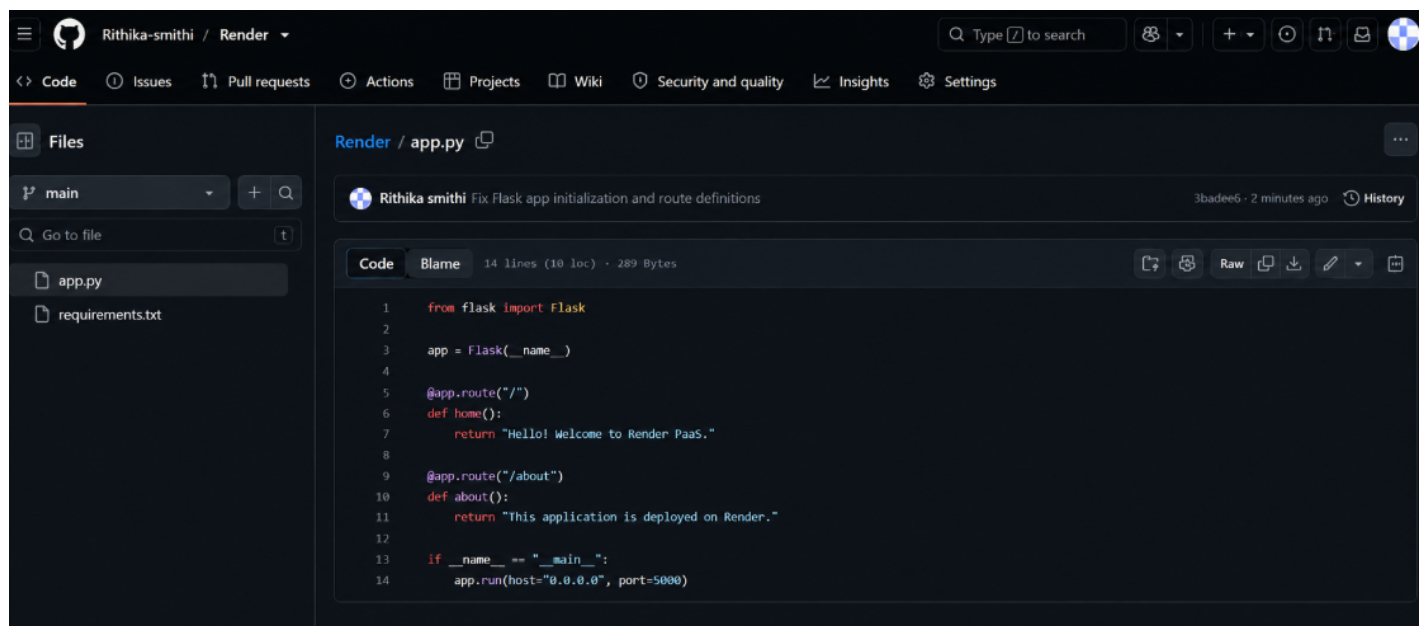
Step 1: Sign in to your GitHub account or create one if you do not already have an account.

Step 2: Create a project folder named RenderHelloApp.

Step 3: Inside the project folder, create the following files:

- app.py
- requirements.txt

Step 4: Write the Flask application code in the app.py file.

The screenshot shows a GitHub repository interface for a user named 'Rithika-smithi'. The repository is named 'Render'. The 'Code' tab is selected, showing the 'app.py' file. The file content is a Python Flask application. The left sidebar shows the file structure with 'main' selected. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security and quality, Insights, and Settings. The file 'app.py' is highlighted in the sidebar. The main content area shows the code for 'app.py' with line numbers 1 through 14. The code imports Flask, creates an app, and defines two routes: '/' and '/about'. The '/' route returns 'Hello! Welcome to Render PaaS.' and the '/about' route returns 'This application is deployed on Render.' The code also includes a standard if __name__ == '__main__': block to run the app on host 0.0.0.0 at port 5000.

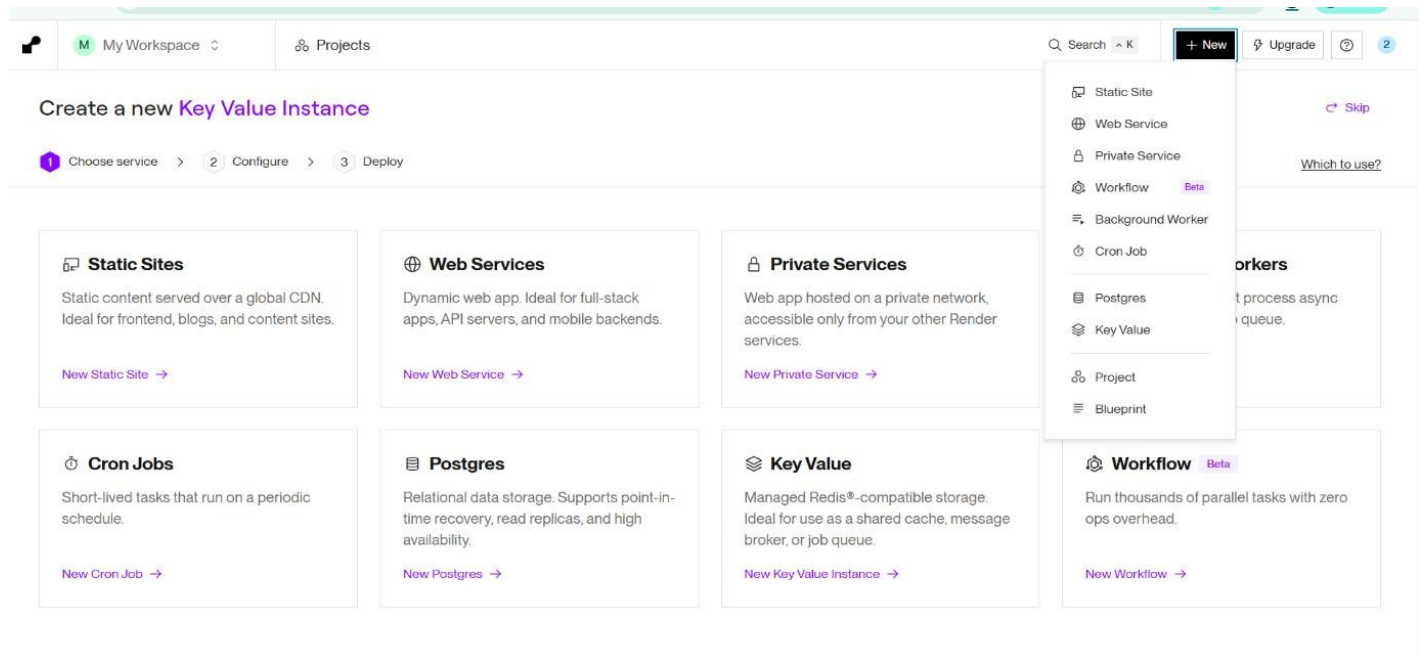
```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def home():
7     return "Hello! Welcome to Render PaaS."
8
9 @app.route("/about")
10 def about():
11     return "This application is deployed on Render."
12
13 if __name__ == "__main__":
14     app.run(host="0.0.0.0", port=5000)
```

Step 5: Add the required Python packages such as Flask and Gunicorn to the requirements.txt file.

Step 6: Create a new public repository called RenderHelloApp on GitHub.

Step 7: Upload both project files to the repository and commit the changes.

Step 8: Create an account on Render and connect it to your GitHub account.



Step 9: Create a new Web Service in Render.

Step 10: Select the RenderHelloApp repository from GitHub.

Step 11: Enter the deployment details:

- Language: Python
- Branch: main
- Root Directory: Leave empty
- Build Command: pip install -r requirements.txt
- Start Command: gunicorn app:app
- Instance Type: Free (if available)

My Workspace

New Web Service

+ New
Upgrade
?
2

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

Build Command

Render runs this command to build your app before each deploy.

Start Command

Render runs this command to start your app with each deploy.

Instance Type

For hobby projects

Free	512 MB (RAM)	Upgrade to enable more features Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.
\$0 / month	0.1 CPU	

For professional use

For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime

Starter	512 MB (RAM)	Standard	2 GB (RAM)
\$7 / month	0.5 CPU	\$25 / month	1 CPU

Pro	4 GB (RAM)	Pro Plus	8 GB (RAM)
------------	------------	-----------------	------------

Step 12: Click Deploy Web Service and wait for the deployment to complete.

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

Generate

+ Add Environment Variable
Add from .env

> **Advanced**

Deploy Web Service

Step 13: After deployment, open the URL generated by Render and check whether the home page is displayed correctly.

Step 14: Add /about at the end of the application URL to verify that the second route is also working properly.

My Workspace

ENVIRONMENT

render rithika

Events

Settings

MONITOR

Logs

Metrics

MANAGE

Environment

Shell

Scaling

Previews

Disks

Changelog

Invite a friend

Contact support

My project / Production / render rithika

Search

+ New

Upgrade

R

WEB SERVICE

render rithika Python 3 Free Upgrade your instance

Connect

Manual Deploy

Service ID: srv-d8i9jtl0e5c73o2n91g

Rithika-smithi / Render main

https://render-rithika.onrender.com

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. Upgrade now

July 30, 2026 at 9:55 PM Live

35fcdede Fix Flask app initialization and route definitions

All logs Search logs Live tail

Jul 30

09:56:05 PM Using cached flask-3.0.3-py3-none-any.whl.metadata (3.2 kB)

09:56:05 PM Collecting gunicorn==22.0.0 (from -r requirements.txt (line 2))

09:56:05 PM Using cached gunicorn-22.0.0-py3-none-any.whl.metadata (4.4 kB)

09:56:05 PM Collecting Werkzeug==3.0.1 (from Flask==3.0.3->-r requirements.txt (line 1))

09:56:05 PM Using cached werkzeug-3.0.1-py3-none-any.whl.metadata (4.0 kB)

requirements.txt app.py 127.0.0.1:5000

http://127.0.0.1:5000/

Hello from Render!

My Flask application is deployed successfully.

Conclusion

This experiment demonstrated how a Flask application can be deployed using Render, a Platform as a Service. I learned that PaaS simplifies application deployment by handling the server setup and hosting, allowing developers to publish applications quickly without managing infrastructure.

Ex: 3

Date:

Setting Up Hybrid Cloud Connectivity Using Cisco Packet Tracer (Simulation)

Aim

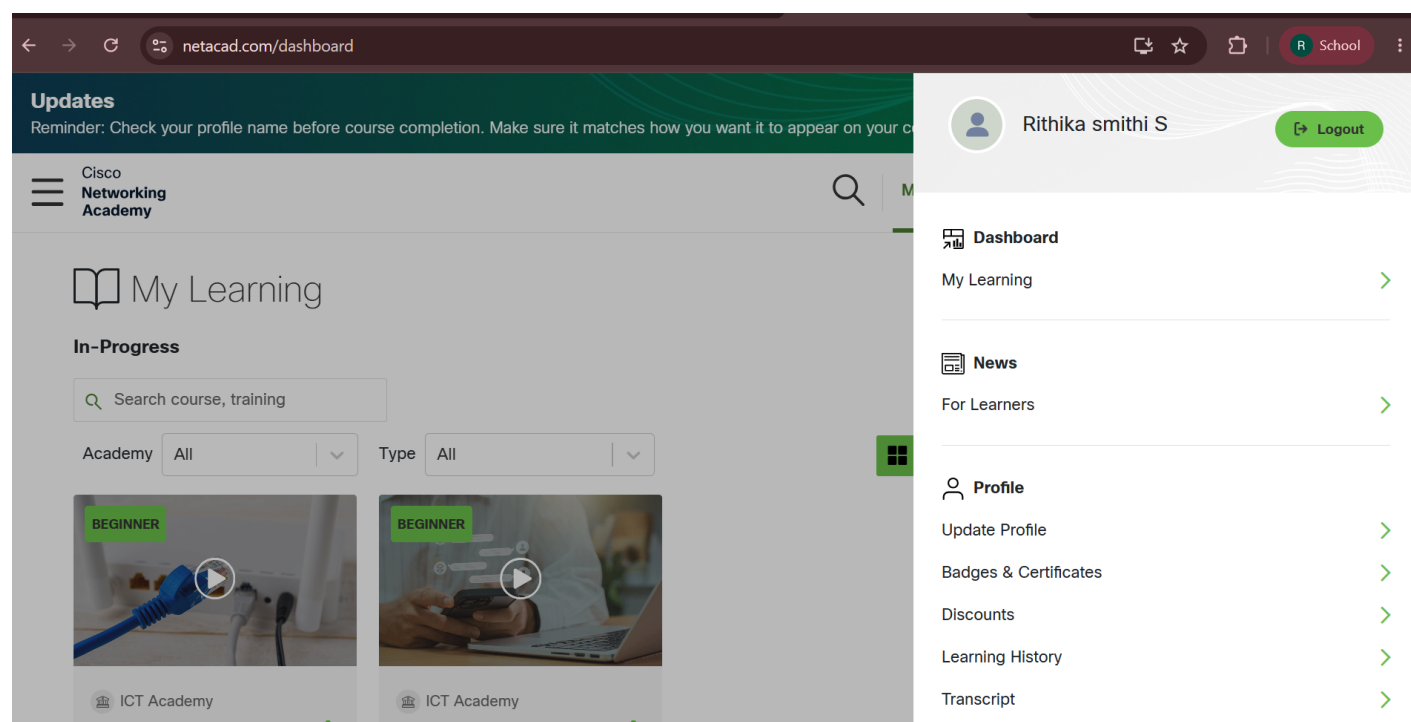
To simulate a secure hybrid cloud connection between an on-premises data center and a cloud network using routers, switches, subnets, and routing in Cisco Packet Tracer.

Procedure

Step 1: Open Cisco Networking Academy. Go to: <https://www.netacad.com>

Step 2: Create a Free Account or Sign in with your Google Account.

1. Click Create an account (or Sign up).
2. Enter: First Name, Last Name, Email Address, Password.
3. Verify your email if prompted.
4. Sign in.



Step 3: Enroll in the Free Packet Tracer Course.

Cisco Networking Academy
Course

Getting Started with Cisco Packet Tracer

This course is part of the **Learning Collections - Cisco Packet Tracer**

Your on-ramp to Cisco Packet Tracer. Get familiar with the simulation environment and download the latest version.

Self-Paced Online
Learn online at your own pace

Instructor-Led
Learn with an academy

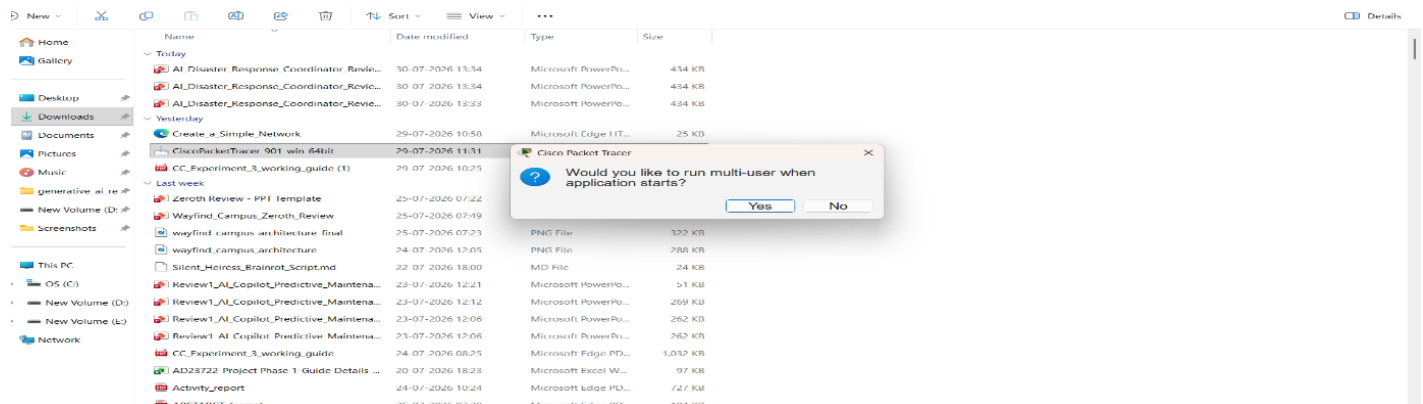
English (English)
Get Started With Self-Paced
15,717,788 already enrolled

AVAILABLE LANGUAGES

العربية, 中文(简体), English, Español, Français, Português, Український

Step 4: Download Packet Tracer. After enrolling:

1. Go to your NetAcad Dashboard.
2. Open the Getting Started with Cisco Packet Tracer course.
3. Navigate to the Resources or Downloads section.
4. Download the version for your operating system: Windows 64-bit (recommended for most PCs), macOS, Linux.



Step 5: Install Packet Tracer.

1. Open the downloaded installer (.exe on Windows).
2. Click Next.
3. Accept the license agreement.
4. Click Next until installation completes.
5. Click Finish.



Step 6: Start Packet Tracer. Open Cisco Packet Tracer from the Start Menu. Sign in with your Cisco NetAcad account or sign in with your google account.

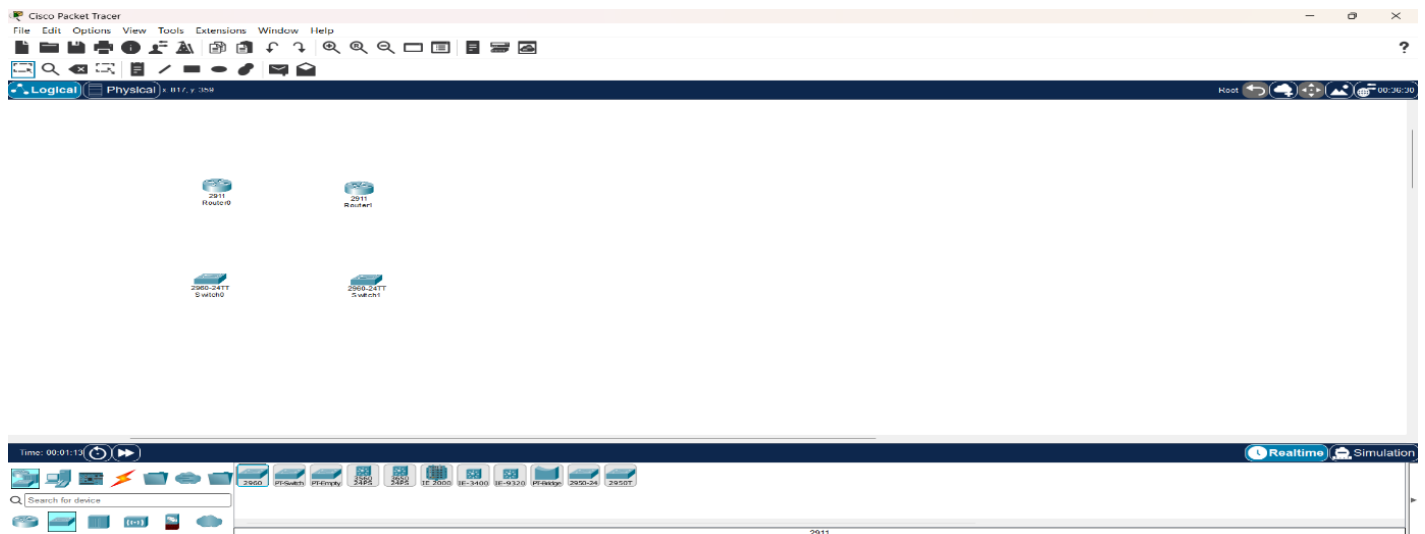
Step 7: Place Two Routers. At the bottom-left of Packet Tracer, you'll see several icons.

1. Click Network Devices (the router icon).
2. Click Routers.
3. Choose 2911 (or 1941 if 2911 isn't available).
4. Click once in the blank workspace to place the first router and second one.



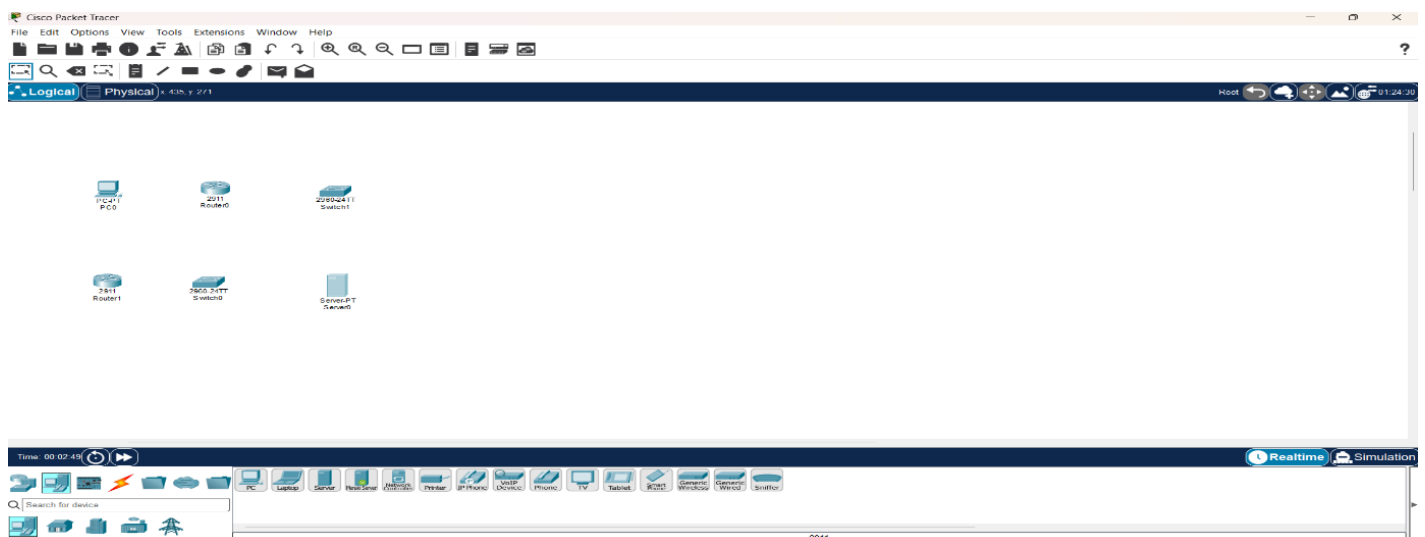
Step 8: Place Two Switches.

1. At the bottom-left, click Network Devices.
2. Click Switches.
3. Select 2960 (for example, 2960-24TT).
4. Place one switch below Router0.
5. Place another switch below Router1.



Step 9: Place One PC and One Server.

1. At the bottom-left, click End Devices (computer icon).
2. Select PC.
3. Place it to the left of Switch0.
4. Select Server.
5. Place it to the right of Switch1.



Step 10: Connect the Devices. At the bottom-left, click the Connections icon (lightning bolt). Choose Copper Straight-Through (usually the first black cable).

Connection 1: PC to Switch0

1. Click PC. 2. Select FastEthernet0. 3. Click Switch0. 4. Select FastEthernet0/1 (or any free FastEthernet port).

Connection 2: Switch0 to Router0 — Using the same Copper Straight-Through cable:

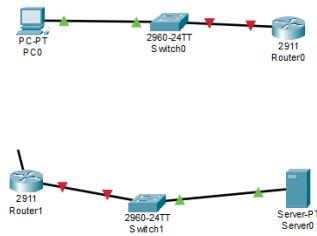
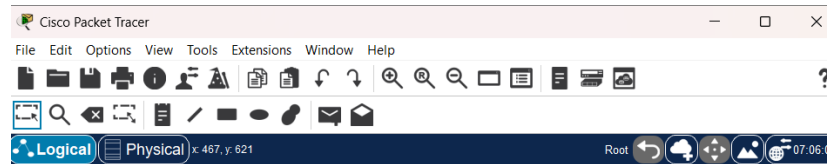
1. Click Switch0. 2. Select FastEthernet0/2. 3. Click Router0. 4. Select GigabitEthernet0/0.

Connection 3: Server to Switch1 — Again use Copper Straight-Through:

1. Click Server. 2. Select FastEthernet0. 3. Click Switch1. 4. Select FastEthernet0/1.

Connection 4: Switch1 to Router1 — Again use Copper Straight-Through:

1. Click Switch1. 2. Select FastEthernet0/2. 3. Click Router1. 4. Select GigabitEthernet0/0.

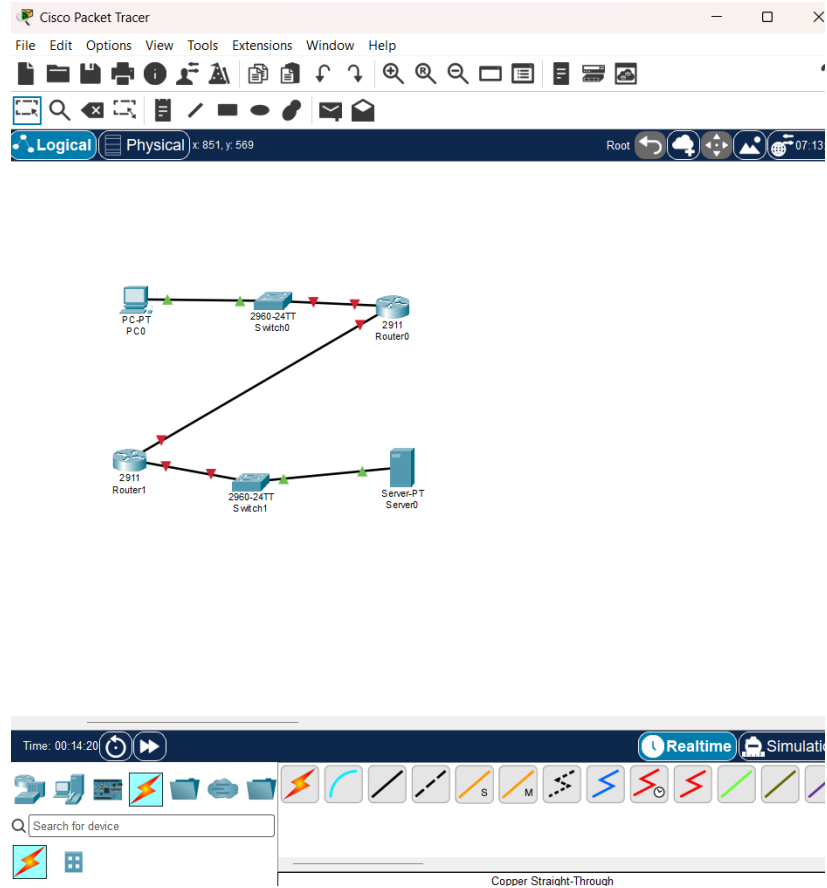


Step 10.1: Connect Router0 to Router1. Now connect the two routers.

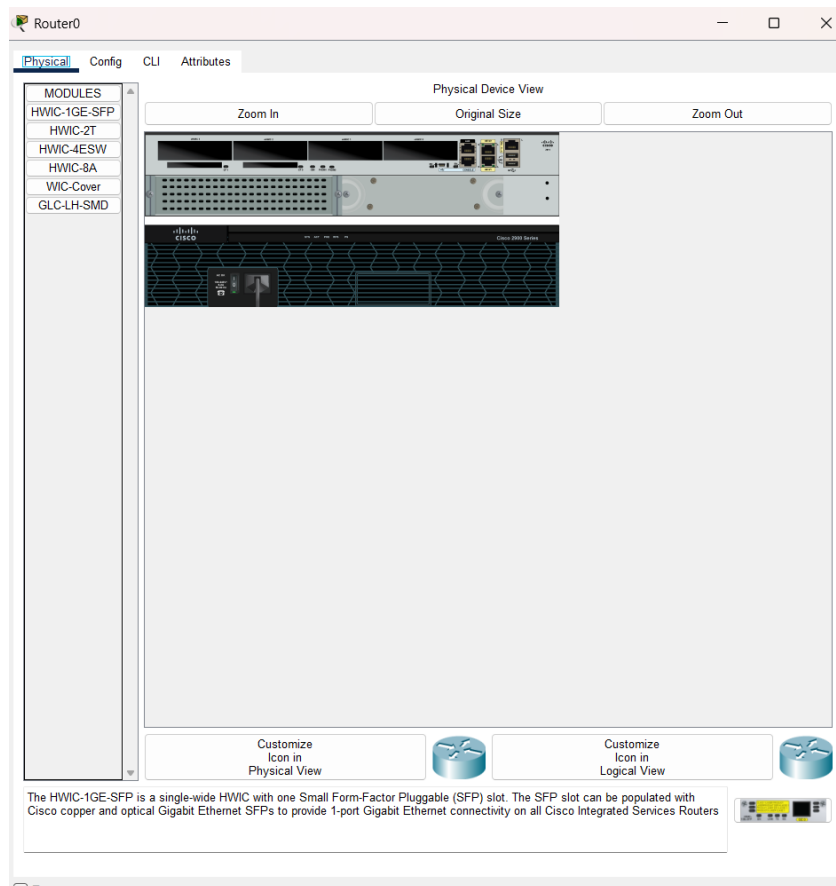
Option A (Recommended) — Use Copper Cross-Over.

1. Click the Connections icon. 2. Select Copper Cross-Over. 3. Click Router0. 4. Select GigabitEthernet0/1. 5. Click Router1. 6. Select GigabitEthernet0/1.

Your topology should now look like this:



Step 10.2: Click the Router. In your workspace, left-click Router0 once.



1. Click the CLI tab.

2. If it asks: "Continue with configuration dialog? [yes/no]:" Type: no. Press Enter. When you see: Router>

```
Router0
Physical Config CLI Attributes
IOS Command Line Interface

smart init is sizing iomem
  TYPE      MEMORY_REQ
  Onboard devices 4
  buffer pools 0x022F6000
  -----
  TOTAL:      0x022F6000
Rounded IOMEM up to: 36Mb.
Using 6 percent iomem. [36Mb/512Mb]

Restricted Rights Legend

Use, duplication, or disclosure by the Government is
subject to restrictions as set forth in subparagraph
(c) of the Commercial Computer Software - Restricted
Rights clause at FAR sec. 52.227-19 and subparagraph
(c) (1) (ii) of the Rights in Technical Data and Computer
Software clause at DFARS sec. 252.227-7013.

cisco Systems, Inc.
170 West Tasman Drive
San Jose, California 95134-1706

Cisco IOS Software, C2900 Software (C2900-UNIVERSALK9-M), Version 15.1(4)M5, RELEASE SOFTWARE (fc2)Technical
Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2007 by Cisco Systems, Inc.
Compiled Wed 18-Jul-07 04:52 by pt_team
Image text-base: 0x2100F918, data-base: 0x24729040

This product contains cryptographic features and is subject to United
States and local country laws governing import, export, transfer and
use. Delivery of Cisco cryptographic products does not imply
third-party authority to import, export, distribute or use encryption.
Importers, exporters, distributors and users are responsible for
compliance with U.S. and local country laws. By using this product you
agree to comply with applicable laws and regulations. If you are unable
to comply with U.S. and local laws, return this product immediately.

A summary of U.S. laws governing Cisco cryptographic products may be found at:
http://www.cisco.com/wml/export/crypto/tool/stqrg.html

If you require further assistance please contact us by sending email to
export@cisco.com.

Cisco CISC02911/K9 (revision 1.0) with 491520K/32768K bytes of memory.
Processor board ID FTX152400K3
3 Gigabit Ethernet interfaces
DRAM configuration is 64 bits wide with parity disabled.
255K bytes of non-volatile configuration memory.
249856K bytes of ATA System CompactFlash 0 (Read/Write)

--- System Configuration Dialog ---
Would you like to enter the initial configuration dialog? [yes/no]:
```

Step 10.3: Configure the LAN Interface (Connected to Switch0). At the Router(config)# prompt, type these commands one at a time, pressing Enter after each line.

interface gigabitEthernet0/0 — You should see: Router(config-if)#

Now type: ip address 192.168.1.1 255.255.255.0 — Press Enter.

Now type: no shutdown — You should see a message similar to: Now

```

Cisco CISC02911/K9 (revision 1.0) with 491520K/32768K bytes of memory.
Processor board ID FTX152400KS
3 Gigabit Ethernet interfaces
DRAM configuration is 64 bits wide with parity disabled.
255K bytes of non-volatile configuration memory.
249856K bytes of ATA System CompactFlash 0 (Read/Write)

--- System Configuration Dialog ---

Would you like to enter the initial configuration dialog? [yes/no]: no

Press RETURN to get started!

Router>configure terminal
      ^
% Invalid input detected at '^' marker.

Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip address 192.168.1.1 255.255.255.0
      ^
% Invalid input detected at '^' marker.

Router(config)#no shutdown
      ^
% Invalid input detected at '^' marker.

Router(config)#Router0
      ^
% Invalid input detected at '^' marker.

Router(config)#prompt
      ^
% Invalid input detected at '^' marker.

Router(config)##prompt
      ^
% Invalid input detected at '^' marker.

Router(config)#interface gigabitEthernet0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

```

Step 10.4: Configure the Router-to-Router Interface. Now type: interface gigabitEthernet0/1

Then: ip address 10.10.10.1 255.255.255.252

Then: no shutdown — Again, wait for the interface to come up.

Finally, type: exit — You should now be back at: Router(config)#

```

Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router(config-if)#exit
Router(config)#interface gigabitEthernet0/1
Router(config-if)#ip address 10.10.10.1 255.255.255.252
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

```

Step 10.5: Verify the Configuration. Type: do show ip interface brief — Since you're in configuration mode, use do before the show command. You should see something similar to:

```

Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#do show ip interface brief
Interface                IP-Address      OK? Method Status          Protocol
GigabitEthernet0/0       192.168.1.1     YES manual up              up
GigabitEthernet0/1       10.10.10.1      YES manual up              up
GigabitEthernet0/2       unassigned      YES unset  administratively down down
Vlan1                    unassigned      YES unset  administratively down down
Router(config)#ip route 10.0.0.0 255.255.255.0 10.10.10.2
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#

```

Next Step: Configure Router1. Click Router1 → CLI. If it says: "Press RETURN to get started!" Press Enter. If it asks: "Continue with configuration dialog? [yes/no]:" Type: no

Then enter these commands one by one: enable; configure terminal; interface gigabitEthernet0/0; ip address 10.0.0.1 255.255.255.0; no shutdown; exit; interface gigabitEthernet0/1; ip address

```

Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gigabitEthernet0/0
Router(config-if)#ip address 10.0.0.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router(config-if)#exit
Router(config)#
Router(config)#interface gigabitEthernet0/1
Router(config-if)#ip address 10.10.10.2 255.255.255.252
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

Router(config-if)#exit
Router(config)#
Router(config)#do show ip interface brief
Interface                IP-Address      OK? Method Status          Protocol
GigabitEthernet0/0       10.0.0.1        YES manual up              up
GigabitEthernet0/1       10.10.10.2      YES manual up              up
GigabitEthernet0/2       unassigned      YES unset  administratively down down
Vlan1                    unassigned      YES unset  administratively down down
Router(config)#

```

Step 10.6: Configure the PC.

1. Click PC. 2. Select Desktop. 3. Click IP Configuration.

Physical Config **Desktop** Programming Attributes

IP Configuration X

Interface FastEthernet0 ▼

IP Configuration

☐ DHCP
 ☒ Static

IPv4 Address 192.168.1.10

Subnet Mask 255.255.255.0

Default Gateway 0.0.0.0

DNS Server 0.0.0.0

IPv6 Configuration

☐ Automatic
 ☒ Static

IPv6 Address /

Link Local Address FE80::20A:41FF:FE7B:342C

Default Gateway

DNS Server

802.1X

☐ Use 802.1X Security

Authentication MD5 ▼

Username

Password

Step 10.7: Configure the Server.

1. Click Server.
2. Select Desktop.
3. Click IP Configuration.

Server0 — □ ×

Physical Config **Services** Desktop Programming Attributes

IP Configuration X

IP Configuration

☐ DHCP
 ☒ Static

IPv4 Address 10.0.0.10

Subnet Mask 255.255.255.0

Default Gateway 10.0.0.1

DNS Server 0.0.0.0

IPv6 Configuration

☐ Automatic
 ☒ Static

IPv6 Address /

Link Local Address FE80::290:CFF:FEDD:9749

Default Gateway

DNS Server

802.1X

☐ Use 802.1X Security

Authentication MD5 ▼

Username

Password

☐ Top

Step 11: Configure Static Routes.

A. Configure Static Route on Router0

1. Click Router0. 2. Go to the CLI tab. 3. If you're at Router#, type: configure terminal

Now enter: ip route 10.0.0.0 255.255.255.0 10.10.10.2

Exit configuration mode: exit — Save the configuration: copy running-config startup-config — When prompted: "Destination filename [startup-config]?" Just press Enter.

B. Configure Static Route on Router1

1. Click Router1. 2. Open the CLI. At the Router# prompt, type: configure terminal

Now enter: ip route 192.168.1.0 255.255.255.0 10.10.10.1

Exit: exit — Save the configuration: copy running-config startup-config — When prompted for the filename, simply press Enter.

```
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#do show ip interface brief
Interface                IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0       192.168.1.1     YES manual up          up
GigabitEthernet0/1       10.10.10.1      YES manual up          up
GigabitEthernet0/2       unassigned      YES unset  administratively down down
Vlan1                    unassigned      YES unset  administratively down down
Router(config)#ip route 10.0.0.0 255.255.255.0 10.10.10.2
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#
```

C. Verify the Route — Type: show ip route

You should see a line similar to: S 192.168.1.0/24 [1/0] via 10.10.10.1 — The S indicates that the static route has been added successfully.

```
Building configuration...
[OK]
Router#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

 10.0.0.0/8 is variably subnetted, 3 subnets, 3 masks
S    10.0.0.0/24 [1/0] via 10.10.10.2
C    10.10.10.0/30 is directly connected, GigabitEthernet0/1
L    10.10.10.1/32 is directly connected, GigabitEthernet0/1
 192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C    192.168.1.0/24 is directly connected, GigabitEthernet0/0
L    192.168.1.1/32 is directly connected, GigabitEthernet0/0

Router#
```

Copy

Paste

Now let's verify Router0. On Router0, at the Router# prompt, type: show ip route

You should see a line like: S 10.0.0.0/24 [1/0] via 10.10.10.2 — If you don't see it, add the route: configure terminal; ip route 10.0.0.0 255.255.255.0 10.10.10.2; end; copy running-config startup-config. Press Enter when it asks for the destination filename.

```
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

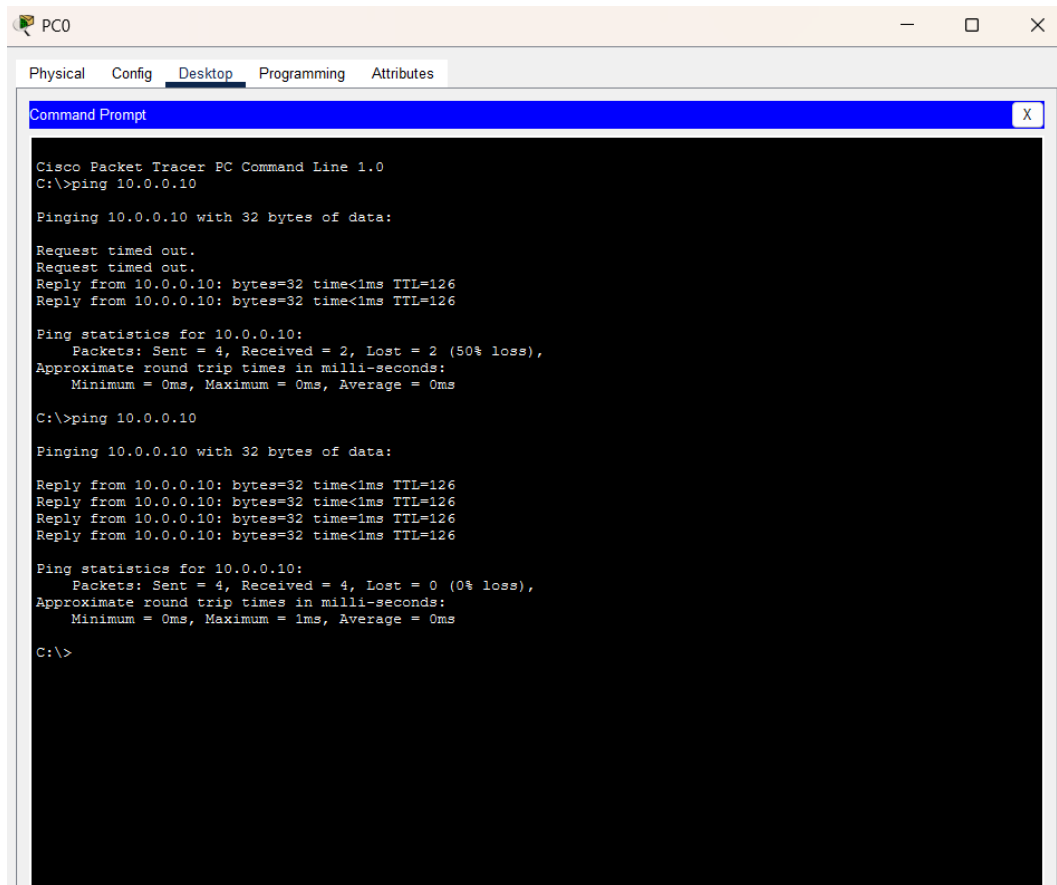
Gateway of last resort is not set

    10.0.0.0/8 is variably subnetted, 4 subnets, 3 masks
C       10.0.0.0/24 is directly connected, GigabitEthernet0/0
L       10.0.0.1/32 is directly connected, GigabitEthernet0/0
C       10.10.10.0/30 is directly connected, GigabitEthernet0/1
L       10.10.10.2/32 is directly connected, GigabitEthernet0/1
S       192.168.1.0/24 [1/0] via 10.10.10.1

Router#
```

Step 12: Test Connectivity. From the PC:

1. Click PC. 2. Go to Desktop → Command Prompt. 3. Type: ping 10.0.0.10 4. Type one more time: ping 10.0.0.10



```
PC0
Physical Config Desktop Programming Attributes
Command Prompt
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 10.0.0.10

Pinging 10.0.0.10 with 32 bytes of data:

Request timed out.
Request timed out.
Reply from 10.0.0.10: bytes=32 time<1ms TTL=126
Reply from 10.0.0.10: bytes=32 time<1ms TTL=126

Ping statistics for 10.0.0.10:
    Packets: Sent = 4, Received = 2, Lost = 2 (50% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 10.0.0.10

Pinging 10.0.0.10 with 32 bytes of data:

Reply from 10.0.0.10: bytes=32 time<1ms TTL=126
Reply from 10.0.0.10: bytes=32 time<1ms TTL=126
Reply from 10.0.0.10: bytes=32 time<1ms TTL=126
Reply from 10.0.0.10: bytes=32 time<1ms TTL=126

Ping statistics for 10.0.0.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>
```

Experiment Successfully Completed! You have successfully simulated Hybrid Cloud Connectivity.

Architecture Diagram of Simulation



Result

A hybrid cloud network was successfully simulated using Cisco Packet Tracer. The on-premises network and cloud network were interconnected through routers configured with static routing. Communication between the PC and the

cloud server was verified successfully using the ping command, demonstrating end-to-end connectivity.

Ex: 4	Virtual Private Server (VPS) Provisioning and Configuration
Date:	

Aim

To provision a Virtual Private Server (VPS) using VirtualBox, install Ubuntu Server, configure network settings (IP Address, Subnet Mask, Gateway, DNS), and enable SSH remote access.

What is VPS?

A Virtual Private Server (VPS) is a software-based server created using virtualization technology. It operates independently with its own operating system, storage, memory, and network configuration while sharing the physical hardware with other virtual machines. Virtualization allows efficient utilization of resources and forms the foundation of cloud computing platforms. Network configuration parameters include:

IP Address

A unique address assigned to a device on a network.

Subnet Mask

Determines the network and host portions of an IP address.

Gateway

Provides access to networks outside the local network.

DNS (Domain Name System)

Converts domain names into IP addresses.

SSH (Secure Shell)

A secure protocol used for remote administration of Linux systems.

Procedure

1. Open the Ubuntu terminal and verify the internet connection using the ping google.com command.

```

valid_lft forever preferred_lft forever
vboxuser@Ubuntu:~/Desktop$ ping google.com
PING google.com (142.251.221.206) 56(84) bytes of data:
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=1 ttl=64 time=35.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=2 ttl=64 time=36.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=3 ttl=64 time=64.2 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=4 ttl=64 time=51.4 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=5 ttl=64 time=60.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=6 ttl=64 time=239 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=7 ttl=64 time=256 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=8 ttl=64 time=93.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=9 ttl=64 time=48.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=10 ttl=64 time=34.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=11 ttl=64 time=55.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=12 ttl=64 time=51.9 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=13 ttl=64 time=33.4 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=14 ttl=64 time=149 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=15 ttl=64 time=246 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=16 ttl=64 time=64.7 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=17 ttl=64 time=81.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=18 ttl=64 time=48.2 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=19 ttl=64 time=44.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=20 ttl=64 time=264 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=21 ttl=64 time=122 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=22 ttl=64 time=50.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=23 ttl=64 time=48.9 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=24 ttl=64 time=52.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=25 ttl=64 time=34.0 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=26 ttl=64 time=56.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=27 ttl=64 time=60.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=28 ttl=64 time=148 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=29 ttl=64 time=45.3 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=30 ttl=64 time=64.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=31 ttl=64 time=277 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=32 ttl=64 time=42.7 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=33 ttl=64 time=36.0 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=34 ttl=64 time=241 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=35 ttl=64 time=301 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=36 ttl=64 time=37.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=37 ttl=64 time=58.2 ms

```

2. Check the network routing information using the ip route command.

```

valid_lft forever preferred_lft forever
vboxuser@Ubuntu:~/Desktop$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.20.20.0/24 dev br-ex proto kernel scope link src 10.20.20.1
vboxuser@Ubuntu:~/Desktop$ cat /etc/resolv.conf

```

3. Verify the DNS (Domain Name System) configuration of the system using the cat /etc/resolv.conf command.

```
vboxuser@Ubuntu:~/Desktop$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .
```

4. Update the Ubuntu package list (if required) using: `sudo apt update`
5. Install the OpenSSH Server package using: `sudo apt install openssh-server -y`
6. Enable and start the SSH service using: `sudo systemctl enable ssh ; sudo systemctl start ssh`
7. Check whether the SSH service is running successfully using: `sudo systemctl status`

```
Processing triggers for dmcc (0.36.2-9ubuntu1)...
```

```
vboxuser@Ubuntu:~/Desktop$ sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
   TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
```

8. Find the IP address of the Ubuntu system using: `hostname -I`
9. Establish an SSH connection to the Ubuntu system using: `ssh <username>@<IP_address>`

```
Search:
vboxuser@Ubuntu:~/Desktop$ hostname -I
10.0.2.15 10.20.20.1 fd17:625c:f037:2:a00:27ff:fe5c:c094
vboxuser@Ubuntu:~/Desktop$ ssh vboxuser@10.0.2.15
The authenticity of host '10.0.2.15 (10.0.2.15)' can't be established.
ED25519 key fingerprint is: SHA256:DPutENGud479D6pbIQjtRgFAhR1uP74ZPyEInGKGpba
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.15' (ED25519) to the list of known hosts.
vboxuser@10.0.2.15's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-28-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

10. Verify the successful SSH login by checking the "Welcome to Ubuntu" message displayed on the terminal.

Conclusion

The Ubuntu system was successfully configured and verified for network connectivity. The OpenSSH Server was installed and enabled, and the SSH service was confirmed to be running. The system's IP address was obtained, and a successful SSH login was performed. Thus, secure remote access to the Ubuntu system was successfully established and verified.

Ex: 5

Date:

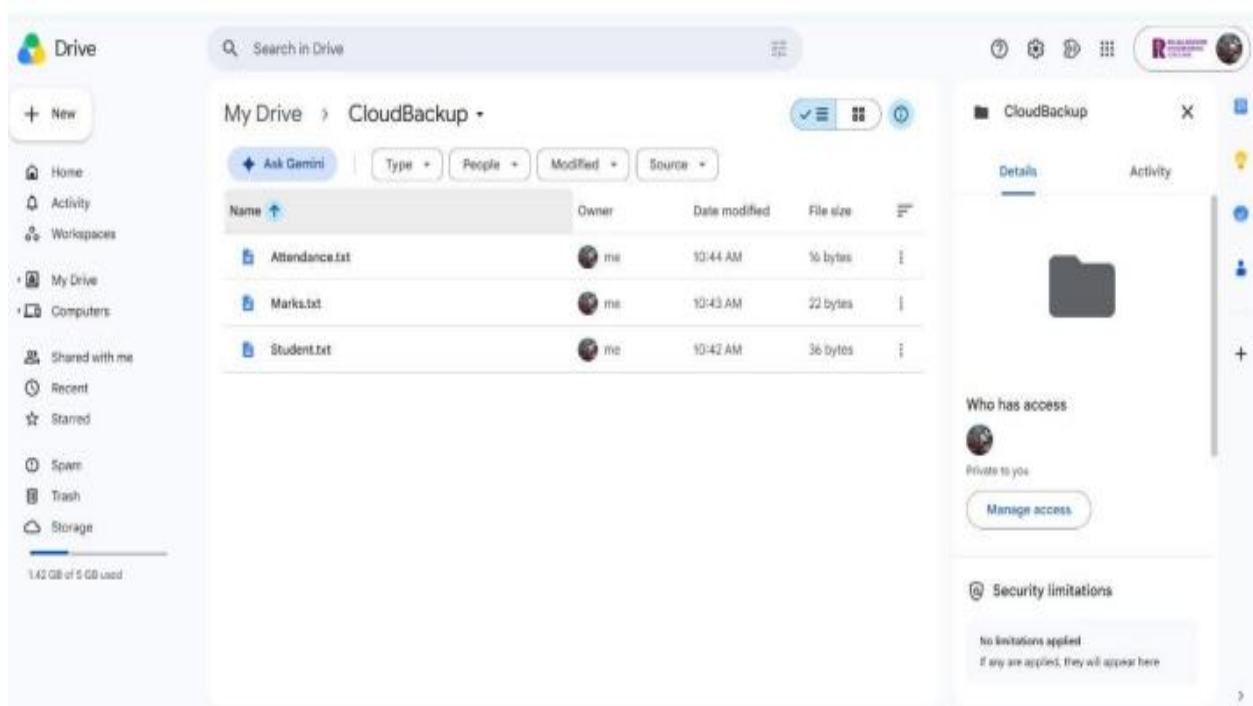
Cloud Backup and Migration using Google Drive

Aim

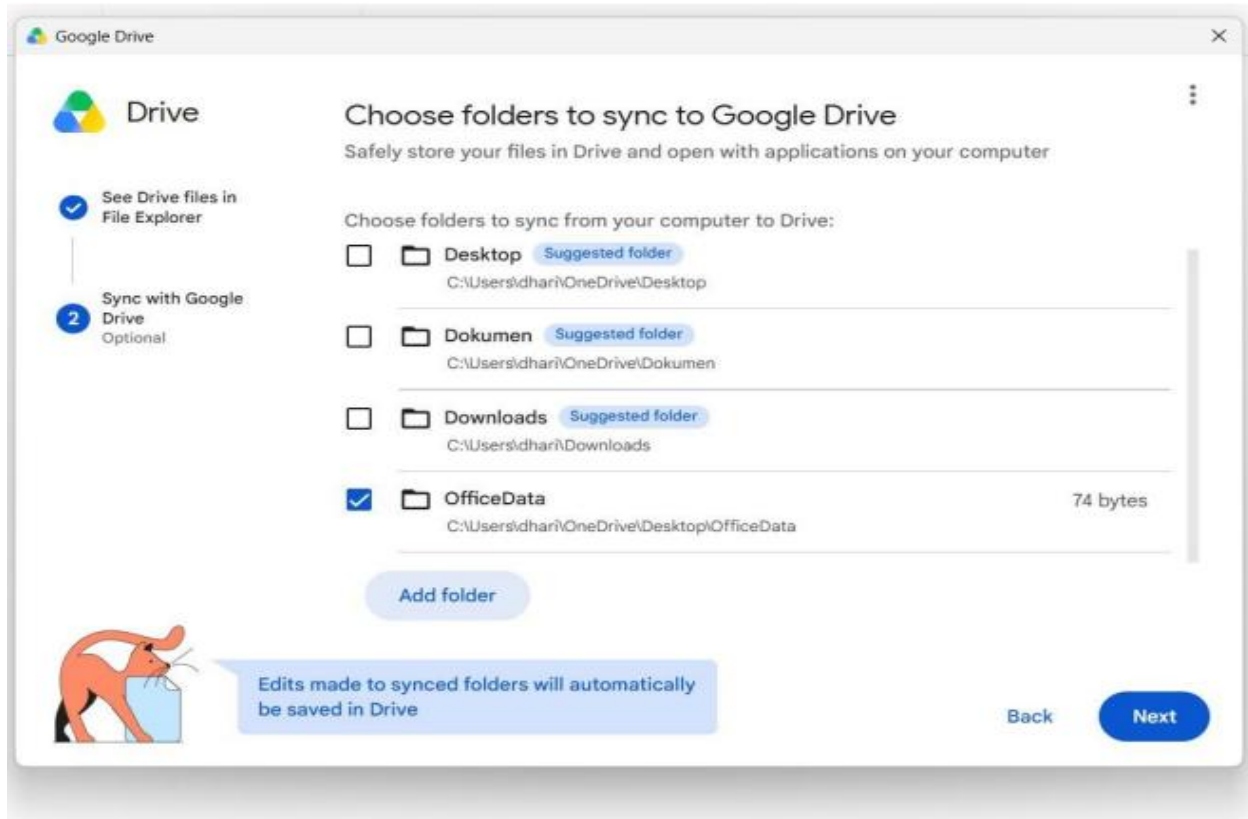
To establish a cloud-based backup solution by utilizing Google Drive, configuring automated synchronization for essential files, and confirming the integrity of the backed-up data.

Procedure

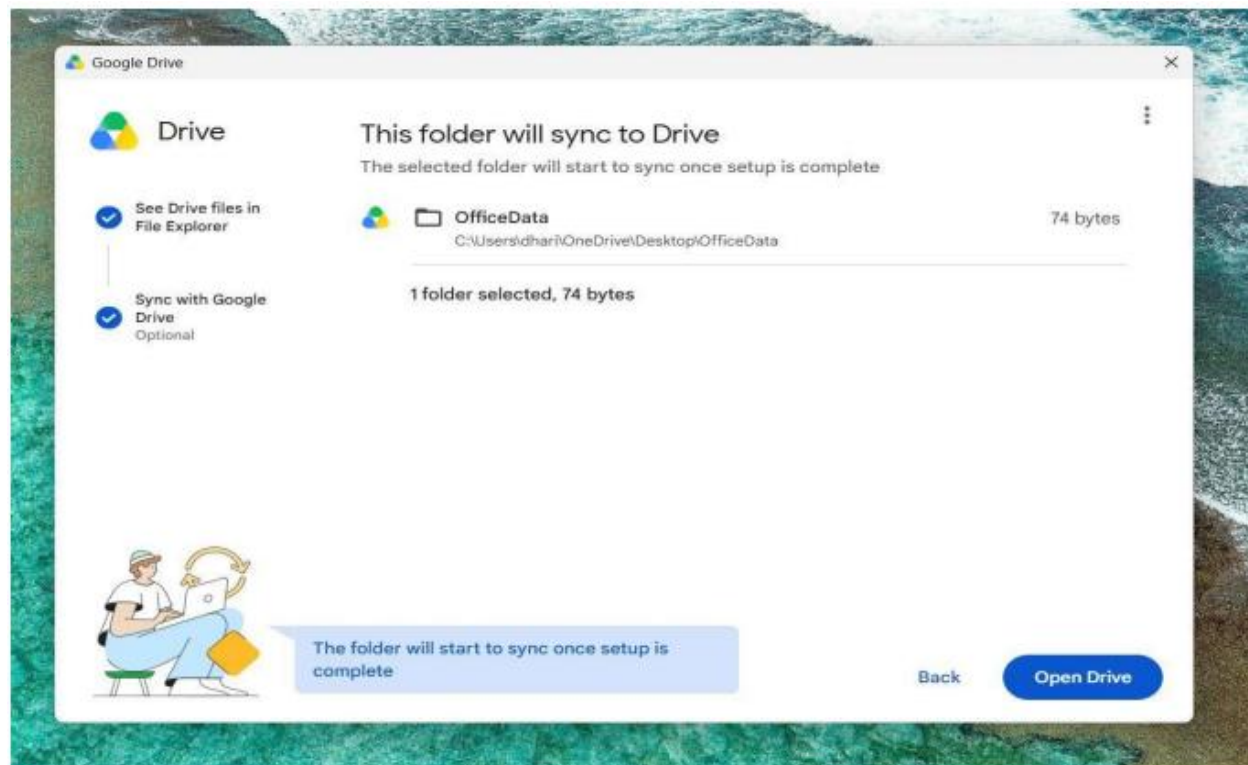
1. Create a local directory designated as "OfficeData" on your workstation.
2. Generate three files named Student.txt, Attendance.txt, and Marks.txt within this directory, populated with example data.
3. Access your Google Drive account via a web browser.
4. Create a dedicated directory titled "CloudBackup" within your Google Drive storage.
5. Upload the three previously created files into this new CloudBackup directory.



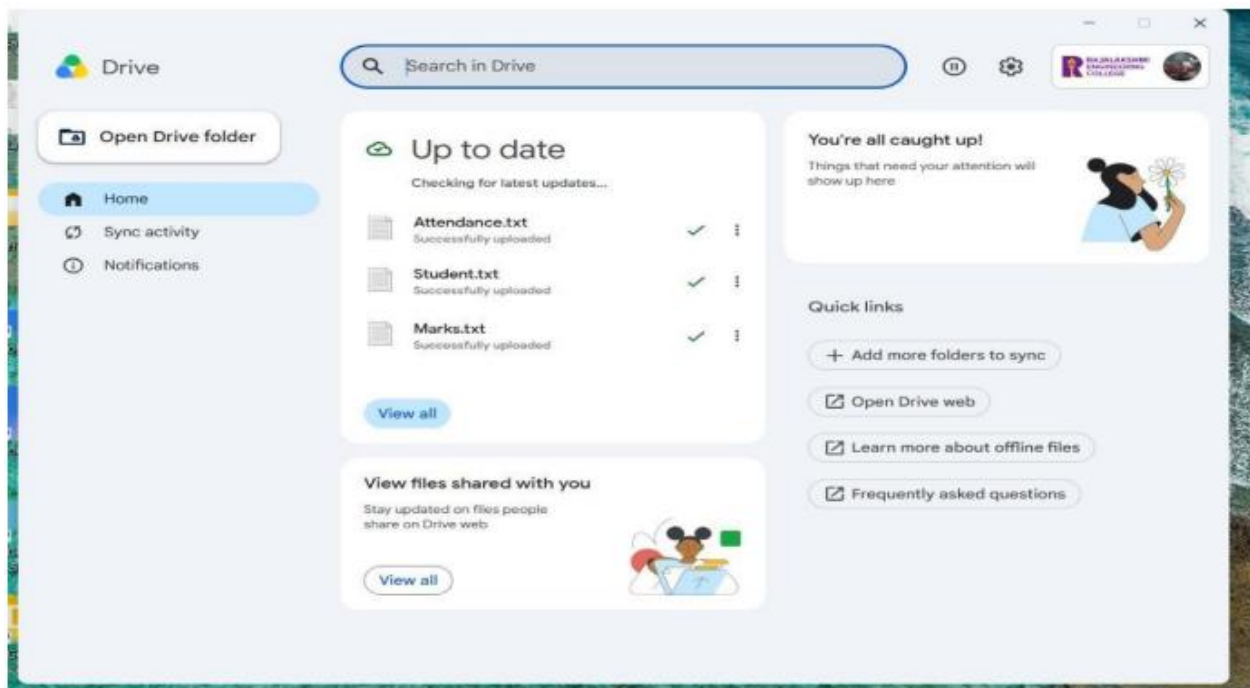
6. Download and install the "Google Drive for Desktop" application, ensuring you sign in with the same credentials used in the browser.
7. Within the Drive for Desktop settings, select the "OfficeData" folder and activate the synchronization feature.



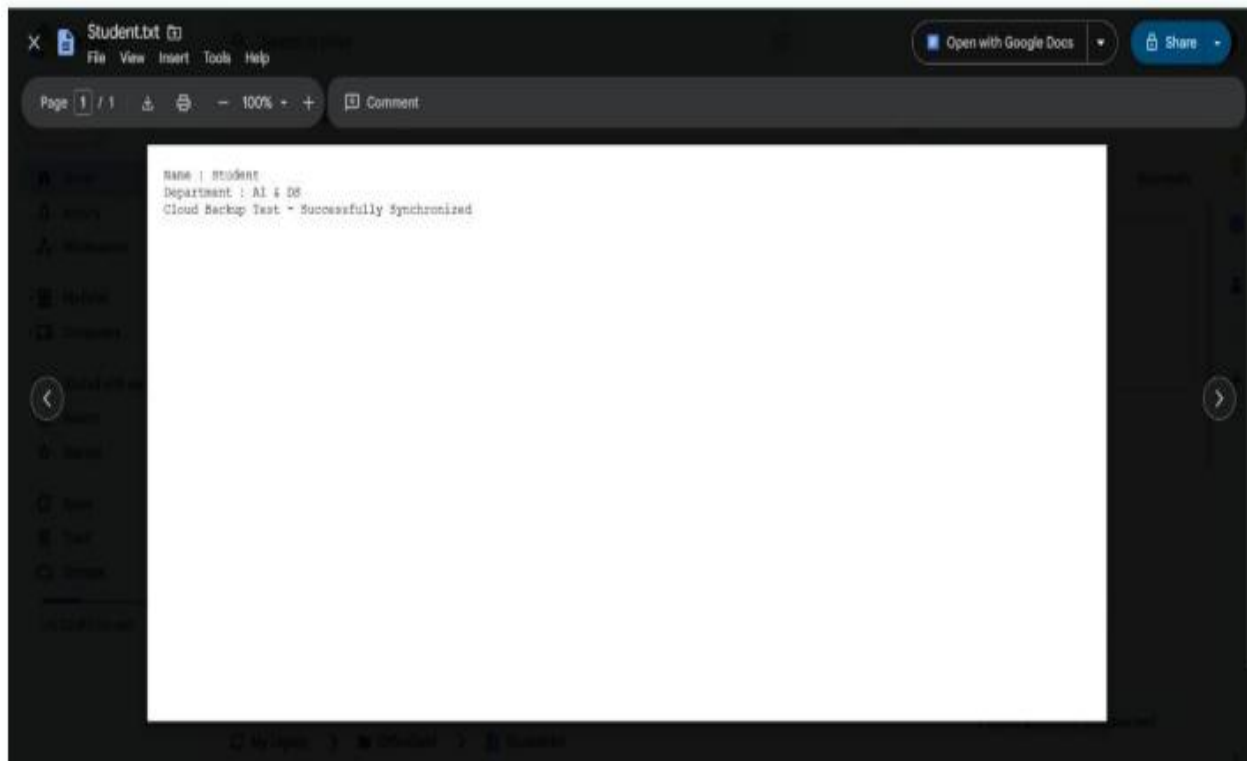
- Verify that the local "OfficeData" folder content now appears within your synced Google Drive files.



- Update Student.txt locally by appending the text "Cloud Backup Test - Successfully Synchronized" and save the changes.
- Monitor the synchronization status in the system tray until it displays "Up to date" or confirms a successful upload.



11. Open the file in Google Drive via the web interface to confirm that the changes made locally are reflected in the cloud version.



Conclusion

A robust cloud backup strategy was successfully deployed using Google Drive. By migrating local files and enabling persistent, automated synchronization, the integrity and availability of the data were verified, effectively demonstrating a reliable cloud-based recovery and backup mechanism.

Ex: 6

Date:

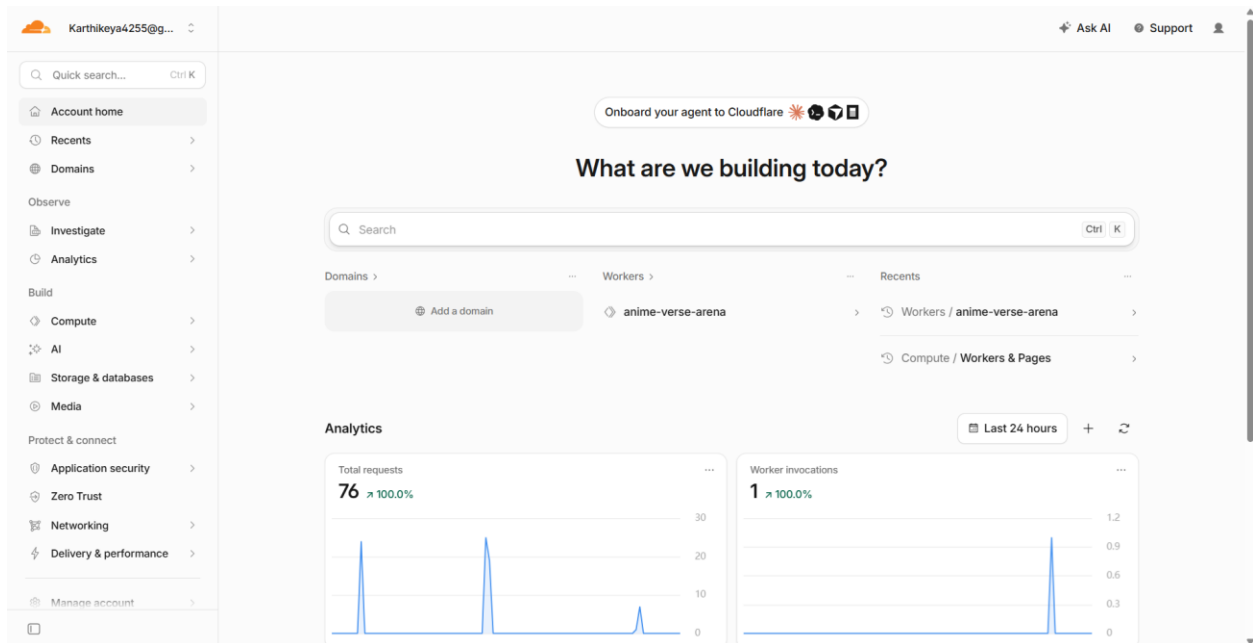
CDN Service Selection and Configuration

Aim

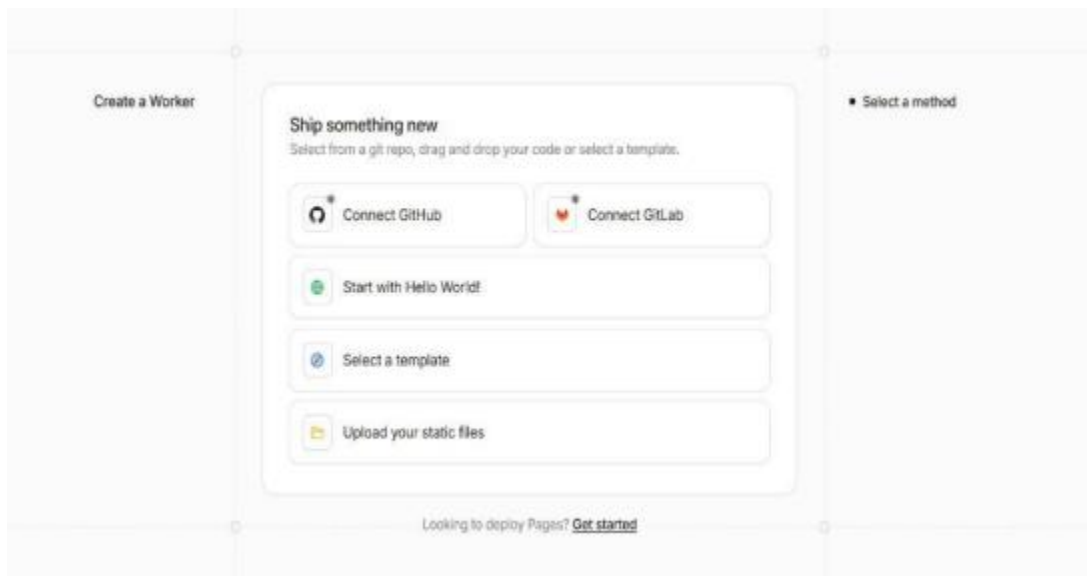
To implement a Content Delivery Network (CDN) by selecting a service provider, deploying a simple web application using Cloudflare, and validating the application's delivery via the edge network by inspecting HTTP response headers using browser developer tools.

Procedure

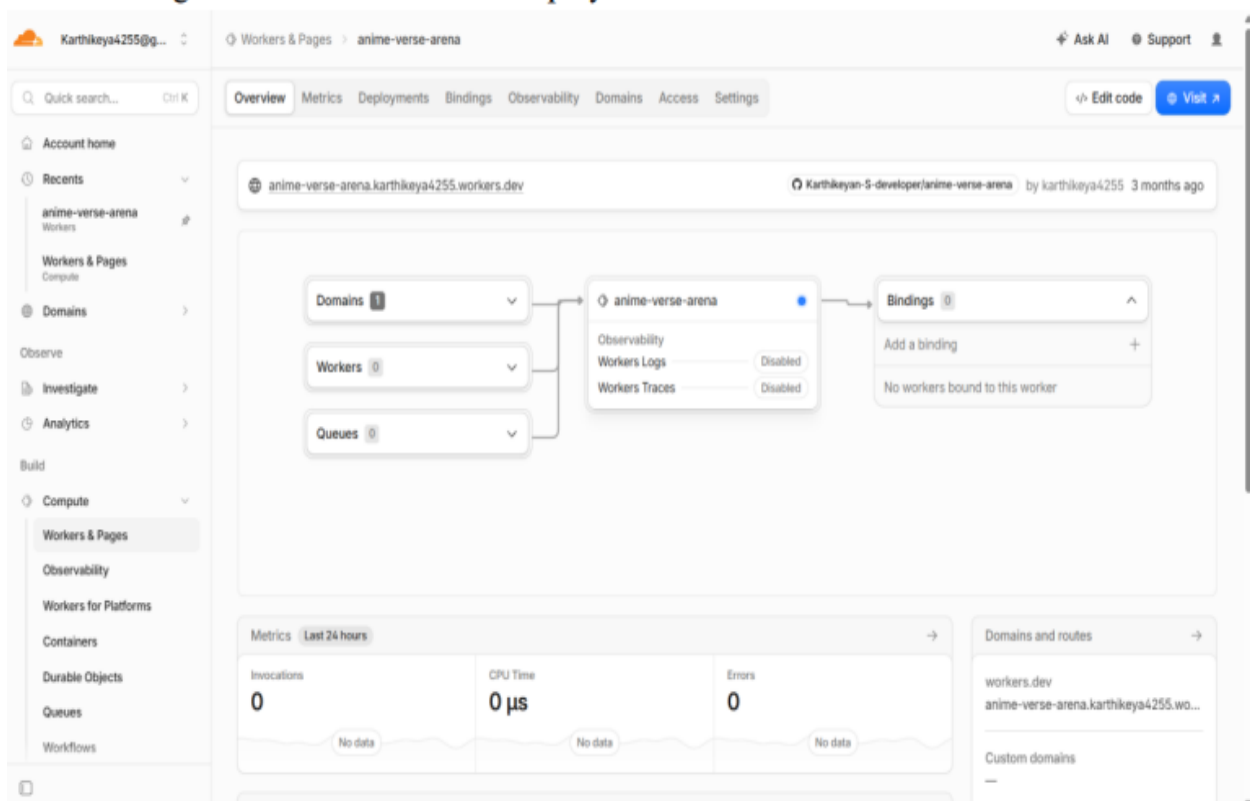
1. Choose a CDN Provider: Cloudflare was selected for its comprehensive suite of edge computing, DNS, security, and CDN capabilities.
2. Access Cloudflare Dashboard: Log in to the Cloudflare management console. No modifications to DNS records or nameservers are required for this deployment.



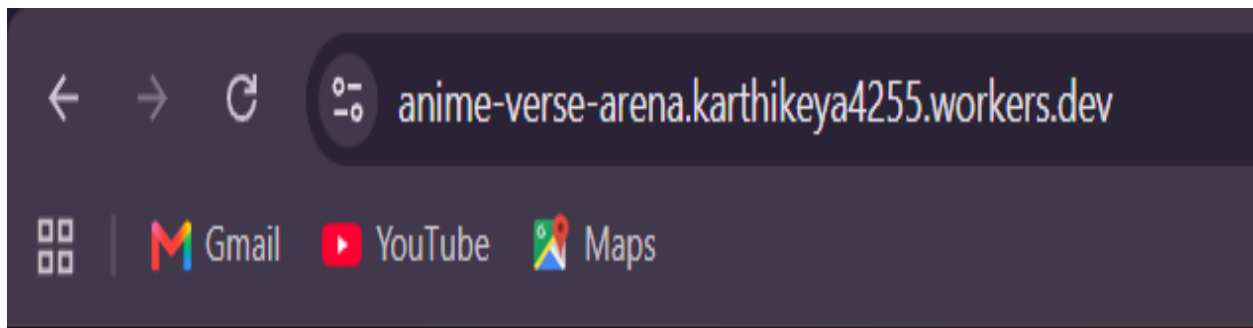
3. Initialize a Worker: Navigate to the Cloudflare dashboard, select the option to create a new application, and choose the created repo to initialize a new Worker.



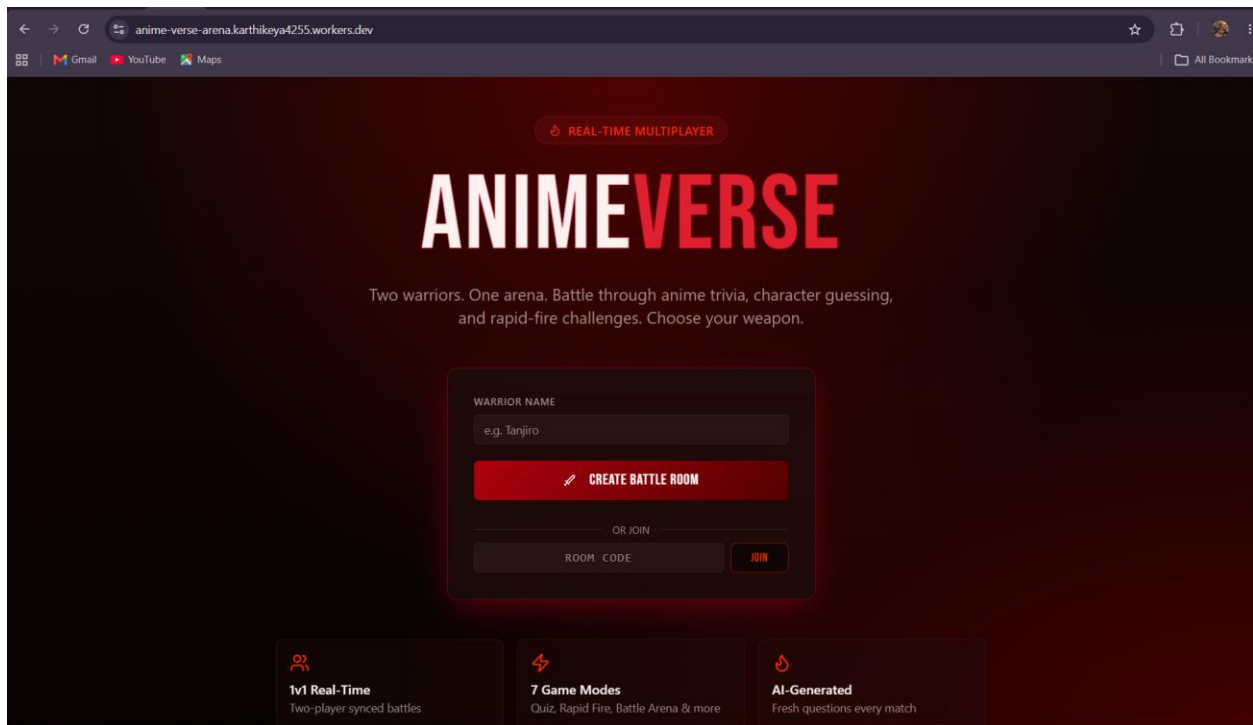
4. Configure the Worker: Retain the default name and the pre-generated repo source code as provided by the platform.
5. Deploy the Application: Click the deployment button and allow Cloudflare to complete the process, which generates a public URL ending in 'workers.dev'.



6. Verify Initial Deployment: Select "Visit" to open the URL in a browser tab and confirm that the website is displayed.



7. Monitor Invocations: Return to the Cloudflare dashboard's analytics section to verify that the invocation count has increased, confirming successful traffic processing.



8. Create Custom Experiment Page: Open the code editor, replace the boilerplate code with a custom HTML structure describing the CDN service experiment, and re-deploy.
9. Test CDN Webpage: Click "Visit" once more to load the customized CDN experiment webpage in your browser.
10. Analyze HTTP Response: Open the webpage, press F12 to access Developer Tools, navigate to the "Network" tab, and refresh the page (Ctrl + R). Select the request associated with the 'workers.dev' URL.
11. Review Response Headers: Within the "Headers" section, inspect the following:
 - Request Method: GET
 - Status Code: 200 OK
 - Server: cloudflare
 - CF-Ray
 - Content-Type: text/html
12. Confirm Successful Delivery: The 200 OK status code and the presence of specific Cloudflare headers verify that the content was successfully routed and served through the Cloudflare edge network.

Conclusion

The experiment successfully demonstrated the deployment of an application via Cloudflare Workers. By generating a public URL and validating the HTTP response headers (specifically verifying the 'Server: cloudflare' and 'CF-Ray' fields), it was confirmed that the application was effectively delivered through Cloudflare's global edge network.